

Docker for Beginners - Complete Tutorial

Table of Contents

- [What are Containers?](#)
- [What is Docker?](#)
- [Why do you need Docker?](#)
- [Docker Commands](#)
- [Creating Docker Images](#)
- [Environment Variables](#)
- [Docker Networking](#)
- [Docker Storage](#)
- [Docker Compose](#)
- [Container Orchestration](#)

What are Containers?

Containers are lightweight, portable units that package applications with all their dependencies, libraries, and configuration files. They share the host operating system's kernel but run in isolated user spaces.

Key Characteristics:

- **Processes:** Isolated application processes
- **Network:** Separate network interfaces
- **Mounts:** Independent file systems

What is Docker?

Docker is a containerization platform that allows you to package applications into containers. It provides tools to build, ship, and run applications consistently across different environments.

Why do you need Docker?

The Matrix from Hell

Traditional application deployment faces several challenges:

- **Compatibility/Dependency issues:** Different environments require different configurations
- **Long setup time:** Complex installation and configuration processes

- **Different Dev/Test/Prod environments:** Inconsistencies between development, testing, and production

Docker Solutions:

- **Containerize Applications:** Package each service with its dependencies
- **Consistent Environments:** Same container runs identically everywhere
- **Isolated Services:** Each container runs independently

Containers vs Virtual Machines

Feature	Containers	Virtual Machines
Size	MB	GB
Boot Time	Seconds	Minutes
Resource Usage	Lower overhead	Higher overhead
Isolation	Process-level	Hardware-level
OS	Shares host kernel	Full OS per VM

Docker Commands

Basic Container Operations

Run a Container

```
bash
docker run nginx
```

- Downloads image if not available locally
- Creates and starts a container

List Containers

```
bash
docker ps      # Running containers
docker ps -a   # All containers (including stopped)
```

Stop a Container

```
bash
docker stop <container_name_or_id>
```

Remove a Container

```
bash  
docker rm <container_name_or_id>
```

Image Operations

List Images

```
bash  
docker images
```

Remove Images

```
bash  
docker rmi <image_name>
```

Note: Remove all dependent containers before removing an image

Pull an Image

```
bash  
docker pull nginx
```

Advanced Run Options

Run with Tag

```
bash  
docker run redis:4.0
```

Interactive Mode

```
bash  
docker run -i <image>    # Interactive  
docker run -it <image>   # Interactive + Terminal
```

Detached Mode

```
bash
```

```
docker run -d <image> # Run in background
```

Port Mapping

```
bash
```

```
docker run -p 80:5000 webapp # Host:Container
```

```
docker run -p 8000:5000 webapp # Multiple instances
```

Volume Mapping

```
bash
```

```
docker run -v /host/path:/container/path <image>
```

Environment Variables

```
bash
```

```
docker run -e APP_COLOR=blue <image>
```

Container Management

Execute Commands in Running Container

```
bash
```

```
docker exec <container> cat /etc/hosts
```

Attach to Running Container

```
bash
```

```
docker attach <container_id>
```

Inspect Container

```
bash
```

```
docker inspect <container>
```

View Container Logs

```
bash
```

```
docker logs <container>
```

Creating Docker Images

Dockerfile Structure

A Dockerfile contains instructions to build a custom image:

```
dockerfile

FROM Ubuntu
RUN apt-get update
RUN apt-get install python
RUN pip install flask
RUN pip install flask-mysql
COPY . /opt/source-code
ENTRYPOINT FLASK_APP=/opt/source-code/app.py flask run
```

Dockerfile Instructions

- **FROM:** Base image
- **RUN:** Execute commands during build
- **COPY:** Copy files from host to image
- **ENTRYPOINT:** Default command when container starts
- **CMD:** Default arguments for entrypoint

Building Images

```
bash

docker build -t my-app .
docker push my-app
```

Layered Architecture

Docker images use a layered approach:

- Each instruction creates a new layer
- Layers are cached for efficiency
- Failed builds resume from last successful layer

CMD vs ENTRYPOINT

CMD

dockerfile

```
FROM Ubuntu
CMD sleep 5
```

- Can be overridden completely
- `docker run ubuntu-sleeper sleep 10`

ENTRYPOINT

dockerfile

```
FROM Ubuntu
ENTRYPOINT ["sleep"]
```

- Cannot be overridden, only appended
- `docker run ubuntu-sleeper 10`

Combined Approach

dockerfile

```
FROM Ubuntu
ENTRYPOINT ["sleep"]
CMD ["5"]
```

- Provides default value that can be overridden

Environment Variables

Setting Environment Variables

In application code:

python

```
import os
color = os.environ.get('APP_COLOR')
```

In Docker:

bash

```
docker run -e APP_COLOR=blue simple-webapp
docker run -e APP_COLOR=green simple-webapp
```

Inspecting Environment Variables

```
bash
```

```
docker inspect <container>
```

Docker Networking

Default Networks

1. **Bridge** (default): Internal private network
2. **Host**: Container uses host network directly
3. **None**: No network access

```
bash
```

```
docker run ubuntu          # Bridge network
```

```
docker run ubuntu --network=none # No network
```

```
docker run ubuntu --network=host  # Host network
```

User-Defined Networks

```
bash
```

```
docker network create \  
  --driver bridge \  
  --subnet 182.18.0.0/16 \  
  custom-network
```

```
docker network ls
```

Embedded DNS

Docker provides built-in DNS resolution:

- Containers can reach each other by name
- DNS server at 127.0.0.11

Docker Storage

File System Structure

```
/var/lib/docker/
├── aufs/
├── containers/
├── images/
└── volumes/
```

Volume Types

Named Volumes

```
bash

docker volume create data_volume
docker run -v data_volume:/var/lib/mysql mysql
```

Bind Mounts

```
bash

docker run -v /host/data:/var/lib/mysql mysql
```

Mount Command

```
bash

docker run \
  --mount type=bind,source=/data/mysql,target=/var/lib/mysql mysql
```

Storage Drivers

- AUFS
- ZFS
- BTRFS
- Device Mapper
- Overlay/Overlay2

Docker Compose

What is Docker Compose?

Docker Compose allows you to define and run multi-container applications using a YAML file.

Sample Application Architecture

voting-app ← → redis ← → worker ← → postgres ← → result-app

Basic docker-compose.yml

```
yaml

version: '3'
services:
  redis:
    image: redis

  db:
    image: postgres:9.4

  vote:
    image: voting-app
    ports:
      - "5000:80"
    links:
      - redis

  result:
    image: result-app
    ports:
      - "5001:80"
    links:
      - db

  worker:
    image: worker
    links:
      - db
      - redis
```

Using Build Instead of Image

```
yaml

vote:
  build: ./vote
  ports:
    - "5000:80"
```

Docker Compose Versions

Version 1 (Legacy)

- No version specification
- All containers on default bridge network

Version 2

- Must specify version
- Automatic network creation
- Dependency management with `depends_on`

Version 3

- Latest version
- Supports Docker Swarm
- Advanced networking features

Networks in Compose

```
yaml

version: '3'
services:
  vote:
    networks:
      - front-end
      - back-end

  redis:
    networks:
      - back-end

networks:
  front-end:
  back-end:
```

Running Compose

```
bash

docker-compose up
```

Docker Registry

Image Naming Convention

```
registry/user/repository:tag
```

Examples:

- `docker.io/nginx/nginx`
- `gcr.io/kubernetes-e2e-test-images/dnsutils`

Default Registry

- Docker Hub (docker.io) is the default public registry

Private Registry

Login to Private Registry

```
bash

docker login private-registry.io
```

Deploy Private Registry

```
bash

docker run -d -p 5000:5000 --name registry registry:2

# Tag and push
docker image tag my-image localhost:5000/my-image
docker push localhost:5000/my-image
docker pull localhost:5000/my-image
```

Container Orchestration

Why Orchestrate?

When you have hundreds of containers:

- **High Availability:** Restart failed containers
- **Load Balancing:** Distribute traffic
- **Scaling:** Add/remove containers based on demand
- **Rolling Updates:** Deploy new versions without downtime

Docker Swarm

Initialize Swarm

```
bash
```

```
# On manager node
```

```
docker swarm init
```

```
# On worker nodes
```

```
docker swarm join --token <token> <manager-ip>
```

Create Services

```
bash
```

```
docker service create --replicas=3 my-web-server
```

```
docker service create --replicas=3 -p 8080:80 my-web-server
```

Kubernetes vs Docker Swarm

Kubernetes Benefits

- More advanced orchestration
- Better scaling capabilities
- Rich ecosystem
- Production-ready features

Basic Kubernetes Commands

```
bash
```

```
kubectl run --replicas=1000 my-web-server
```

```
kubectl scale --replicas=2000 my-web-server
```

```
kubectl rolling-update my-web-server --image=web-server:2
```

Docker Engine Architecture

Components

- **Docker Daemon:** Background service
- **REST API:** Interface for communication
- **Docker CLI:** Command-line interface

Remote Access

```
bash
```

```
docker -H=remote-docker-engine:2375 run nginx
```

```
docker -H=10.123.2.1:2375 run nginx
```

Containerization Concepts

Namespaces

Docker uses Linux namespaces for isolation:

- **Process ID (PID):** Process isolation
- **Network:** Network isolation
- **Mount:** File system isolation
- **User:** User isolation
- **UTS:** Hostname isolation
- **IPC:** Inter-process communication isolation

Control Groups (cgroups)

Limit resource usage:

```
bash
```

```
docker run --cpus=.5 ubuntu    # Limit CPU
```

```
docker run --memory=100m ubuntu # Limit memory
```

Docker on Different Platforms

Windows

1. **Docker Toolbox:** Uses VirtualBox (legacy)
2. **Docker Desktop:** Native Windows containers

Windows Container Types

- **Windows Server Containers:** Shared kernel
- **Hyper-V Containers:** Isolated kernel

macOS

1. **Docker Toolbox:** Uses VirtualBox (legacy)
2. **Docker Desktop:** Native containerization

Best Practices

Security

- Use official base images
- Keep images updated

- Don't run as root
- Scan for vulnerabilities

Performance

- Use multi-stage builds
- Minimize layers
- Use .dockerignore
- Optimize image size

Production

- Use health checks
- Implement logging
- Monitor resource usage
- Plan for scaling

Summary

Docker revolutionizes application deployment by:

- **Solving environment consistency issues**
- **Simplifying application packaging**
- **Enabling microservices architecture**
- **Facilitating DevOps practices**

Key concepts to remember:

- Containers share the host OS kernel
- Images are built in layers
- Docker Compose manages multi-container applications
- Orchestration tools manage containers at scale
- Understanding networking and storage is crucial for production use

This tutorial covers the fundamentals needed to start working with Docker effectively in development and production environments.